Scalar SMU: A RISC-V-Integrated Recurrence Accelerator for Online Attention Merge

Xiaoteng Wang

***Abstract*—Blockwise attention reduces intermediate storage by combining tile-local results into a running normalization state. Its online merge consists of a serial scalar recurrence followed by a regular vector update; the two operations place different demands on a programmable vector processor. This paper presents Scalar SMU, a recurrence-oriented hardware unit integrated into a Snitch/Spatz RISC-V vector cluster. A row-sequential datapath updates the running maximum and normalizer and generates two rescaling weights, while the existing RISC-V Vector (RVV) unit updates the output. A shared exponential lookup table with winner-path bypass, parallel normalization products, and time-multiplexed weight multiplication organize the scalar arithmetic. Two custom instructions separate persistent execution-context configuration from tile-level recurrence, allowing software to retain control of vector dimensions and scheduling. Across nine RTL workloads, the instruction-controlled SMU achieves 5.14–12.08× inclusive-merge speedup and 1.024–1.238× speedup in the measured attention-kernel window, including one-time configuration, against software recurrence plus the same RVV update. A matched full-offload ablation exposes the cost of serial vector traffic, while scalar-only full-cluster synthesis shows a 1.61% mapped standard-cell area increment, excluding SRAM macros. The results demonstrate the effectiveness of specializing the dependent recurrence while reusing existing vector execution resources.**

***Index Terms*—RISC-V, online attention, online merge, accelerator microarchitecture, custom instructions, vector processing.**

I. Introduction

Transformer attention maps query–key similarity scores into weights for value aggregation [1]. In tiled implementations, attention kernels repeatedly normalize scores and accumulate value vectors. Materializing the full attention matrix imposes considerable storage and memory-traffic costs; online normalization and tiled attention avoid this intermediate by merging partial results as computation proceeds [2], [3], [4]. Recent attention kernels further optimize parallel work assignment and precision-dependent execution [5]. The merge requires a running maximum, a normalization sum, and an output vector. Its scalar state transition depends on the outcome of the previous arithmetic stage, whereas the associated vector update contains independent operations across the output dimension.

This asymmetry is relevant to a RISC-V vector cluster. Existing RVV lanes and vector load/store hardware already provide substantial throughput for the regular output update, but the maximum–exponential–normalization chain remains a sequence of scalar instructions and intermediate state accesses. The issue becomes more prominent as vector execution removes the regular arithmetic from the scalar instruction stream: the residual recurrence still requires dependent control, nonlinear arithmetic, and scalar-state communication with memory. Its cost is repeated once per active query row for every subsequent key tile, regardless of the output width. When repeated across query rows and key tiles, this serial work remains visible even in a system equipped with vector arithmetic. Fully duplicating the vector update in a dedicated accelerator would introduce another vector execution and memory-transfer path, with hardware costs and data movement that must be considered at cluster level. The architectural question is therefore how to accelerate the dependent part of the merge using the resources already present in the cluster.

Prior work has optimized online Softmax arithmetic and precision in Softermax [6] and accelerated nonlinear functions within programmable RISC-V systems. SoftEx introduces specialized Softmax/GELU hardware in a heterogeneous cluster [7], whereas VEXP extends the Snitch floating-point execution path with exponentiation-oriented instructions [8]. These approaches provide useful arithmetic specialization, while the state transition *between* attention tiles poses a distinct recurrence-and-vector partitioning problem. Snitch provides the scalar execution substrate [9], and Spatz supplies the compact RVV fabric in which such a partition can be studied [10]. The integration of specialization into an existing programmable platform is also an established concern in accelerator evaluation [11]; related RISC-V vector processors emphasize data-parallel arithmetic [12].

This paper presents *Scalar SMU*, a cluster-integrated unit for online-merge state recurrence. Its contributions are summarized below.

1) Recurrence microarchitecture: A resource-shared scalar datapath computes the updated normalization state and merge weights, while existing RVV lanes perform vector updates.

2) Tile-granular RISC-V integration: Two custom instructions separate persistent execution-context configuration from tile-level recurrence execution. The interface enables repeated merge operations with retained state addresses, processor-visible completion, and software-managed RVV updates.

3) Experimental characterization: RTL workload scaling, matched selective/full-offload experiments, and mapped-area results quantify performance and hardware cost.

II. Online Attention Merge

Let $Q \in \mathbb{R}^{N_q \times d_k}$, $K \in \mathbb{R}^{N_k \times d_k}$, and $V \in \mathbb{R}^{N_k \times d_v}$. The score for query i and key j is $s_{ij} = \mathbf{q}_i^{\mathsf{T}} \mathbf{k}_j / \sqrt{d_k}$. For a

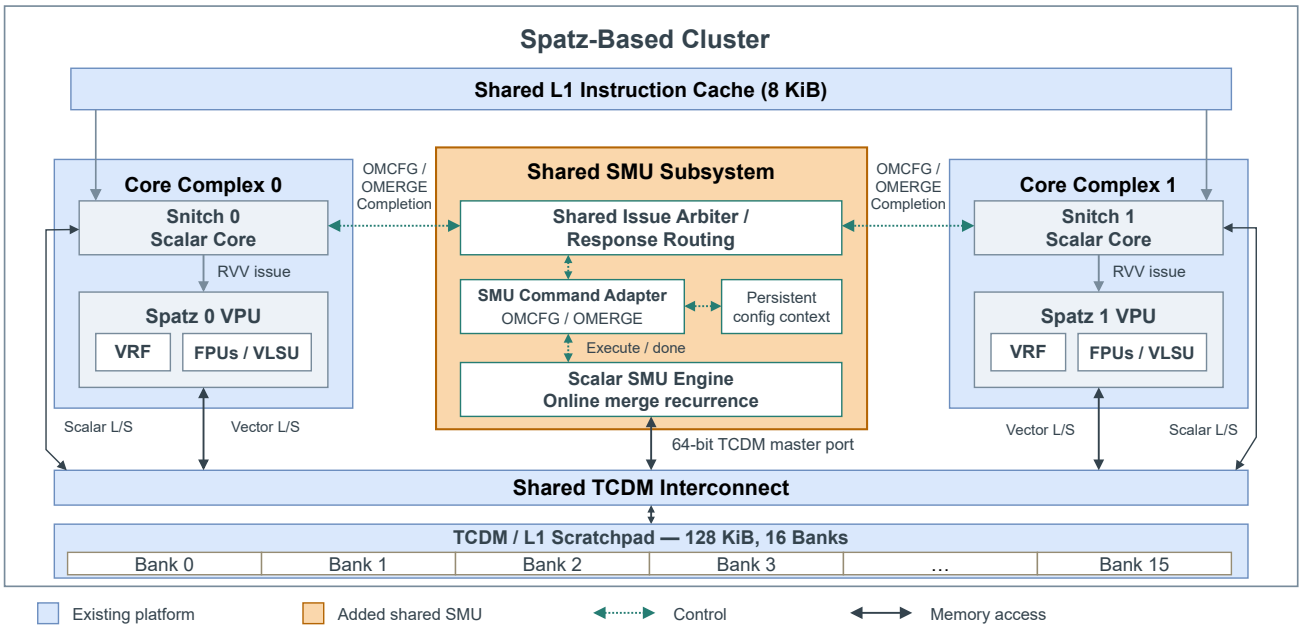


Fig. 1. Cluster-level SMU integration. Snitch dispatches OMCFG/OMERGE through shared issue arbitration and runs RVV instructions on Spatz. The SMU and RVV access separate TCDM master interfaces; running states and merge weights reside in shared memory. Dashed paths denote commands and control.

nonempty set $\mathcal{J}$ of processed keys, the numerically stabilized running state is

$$\begin{aligned} m_i &= \max_{j\in\mathcal{J}} s_{ij}, \\ \ell_i &= \sum_{j\in\mathcal{J}} e^{s_{ij}-m_i}, \\ \mathbf{o}_i &= \ell_i^{-1} \sum_{j\in\mathcal{J}} e^{s_{ij}-m_i}\mathbf{v}_j. \end{aligned} \tag{1}$$

For a new key tile $\mathcal{T}$, define its local state $(m_{i,t}, \ell_{i,t}, \mathbf{o}_{i,t})$ by Eq. (1) over $\mathcal{T}$. The combined scalar state is

$$\begin{aligned} m_i' &= \max(m_i, m_{i,t}), \\ \alpha_i &= e^{m_i - m_i'}, \qquad \beta_i = e^{m_{i,t}-m_i'}, \\ \ell_i' &= \alpha_i \ell_i + \beta_i \ell_{i,t}. \end{aligned} \tag{2}$$

The two normalized partial outputs are then merged as

$$\begin{aligned} w_{i,o} &= \frac{\alpha_i \ell_i}{\ell_i'}, \qquad w_{i,t} = \frac{\beta_i \ell_{i,t}}{\ell_i'}, \\ \mathbf{o}_i' &= w_{i,o}\mathbf{o}_i + w_{i,t}\mathbf{o}_{i,t}. \end{aligned} \tag{3}$$

The first tile initializes the running state, and subsequent tiles apply Eqs. (2)–(3) using bounded-precision scalar arithmetic. Because the output in Eq. (3) maintains running normalization, no final division is required. Carrying only the scalar maximum and normalizer across tiles leaves the normalized output as a software-managed vector updated using the two weight arrays generated by the SMU.

With key tile size B, $T = \lceil N_k/B \rceil$ tiles require $N_q(T-1)$ row-wise scalar recurrences and $N_q d_v(T-1)$ element updates after initialization. Each row's two scalar weights are reused for all d_v output elements. The recurrence in Eq. (2) involves a dependent maximum, nonlinear rescaling, accumulation, and reciprocal; after both weights are available, the operations in Eq. (3) are independent across the output elements. This motivates a compact scalar datapath for $(m_i', \ell_i', w_{i,o}, w_{i,t})$ and the reuse of RVV for $\mathbf{o}_i'$. The implemented engine processes rows serially within one tile command, while software controls B, the tile order, and the vector update.

III. Scalar SMU Microarchitecture

A. Cluster Integration and State Flow

Fig. 1 shows the SMU as a shared cluster-level accelerator, connected to Snitch request routing and to the 128-KiB, 16-bank tightly coupled data memory (TCDM). Spatz retains its vector registers, floating-point lanes, and RVV load/store execution. The SMU maintains two alternating pairs of running maximum and normalizer arrays in TCDM; each tile supplies its local statistics. For each row, the engine reads four FP32 scalars and writes the updated maximum, normalizer, and two merge weights. Software then loads the weights and updates the output vector using RVV, with strip-mining as required by its configured vector length.

The SMU has one TCDM request interface and a finite-state controller that sequences four state reads, scalar calculation, three weight-generation phases, and four writes per row. Address-generation and row counters advance only after successful memory transactions. This row-sequential schedule limits per-row hardware replication; the A/B state-array selector changes following successful command completion. The state arrays are allocated by software, with the persistent context storing their base addresses rather than embedding them in the instruction. The scalar source/destination selection follows a two-buffer discipline: when a command completes, the newly written state becomes the running state for the next tile. The weight arrays provide a compact connection between the scalar engine and the independently scheduled

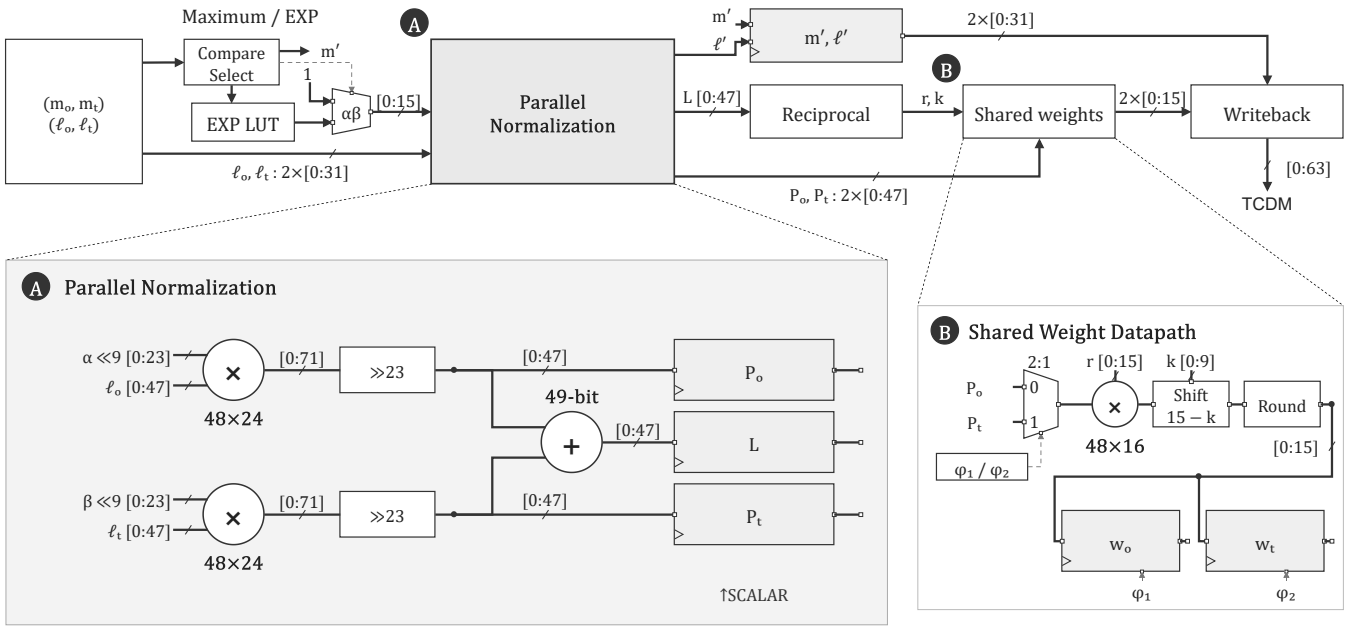


Fig. 2. Scalar SMU datapath. The top-level path generates the updated state and weights; inset A exposes two parallel normalizer products and the shared adder, while inset B shows time-multiplexed weight generation through one multiplier. Maximum selection bypasses the EXP LUT on the winning path. Solid lines indicate data and dashed lines indicate control or inset links. The weight datapath is scheduled across reciprocal evaluation and two successive shared-multiplier phases.

RVV routine. The scalar and vector paths share TCDM but retain separate memory access and execution responsibilities.

B. Arithmetic Datapath and Numerical Formats

Fig. 2 details the implemented recurrence logic. FP32 maximum values are rounded to FP16 before comparison. The winning maximum has unit exponential scaling; the losing branch uses one 256-entry Q1.14 exponential LUT indexed by the FP16 difference, with values below -8 saturated to zero. Both normalizers are converted to unsigned Q16.32 and scaled by parallel 48×24 fixed-point multipliers. A 49-bit sum produces the new normalizer. Its FP16-rounded denominator enters a shared Q1.15 reciprocal LUT with interpolation; a single 48×16 multiplier, time-shared across two phases, forms the normalized old/tile weights. Updated scalar state and FP16-rounded, FP32-stored weights preserve the memory-visible FP32 interface. The two exponential paths share one LUT and the two weights share reciprocal and multiplication resources. The source maximum selection also supplies the control for the unity/exponential multiplexer; no second exponential datapath is required. The weighted normalizer terms are generated concurrently, followed by one wide accumulation. The normalization products and their sum are captured in the SCALAR phase. The following three weight-generation phases evaluate and register the reciprocal parameters (φ_0), then reuse a single multiplier to generate the old-state and tile-state weights (φ_1 and φ_2), respectively. These resource decisions follow the intra-row precedence in Eq. (2), while the D-element arithmetic remains outside the unit.

Table-based nonlinear-function approximation provides a means of trading numerical precision for specialized combinational logic [13]. Here the precision reduction is applied at the recurrence stages where intermediate values enter the exponential, normalization, and reciprocal hardware, with FP32 memory-visible state. This stage-dependent precision assignment also controls the physical cost of the recurrence. These stage-specific bitwidths are fixed at synthesis time.

TABLE I
CUSTOM INSTRUCTION SEMANTICS AT THE SMU INTERFACE.

Instruction	Architectural action
`OMCFG`	Write a selected persistent context field from $X[rs1]$; INIT commits the context.
`OMERGE`	Merge N_q rows for one tile; return completion status in rd.

C. Tile-Level RISC-V Instruction Interface

The custom instructions `OMCFG` and `OMERGE` expose the scalar state transition to software using the RISC-V custom-opcode space [14]. `OMCFG` writes one of nine persistent execution-context fields, including A/B scalar-state addresses, tile-state and weight addresses, and N_q. An INIT operation commits that context and initializes the state selector (Table I). In the instruction encoding, `OMCFG` uses a configuration identifier to select the context field and an integer source register to provide its value; `OMERGE` carries the architectural completion destination. `OMERGE` executes the configured recurrence over all query rows of the next tile, returning a completion status through the accelerator response path. Successful completion swaps the A/B state roles. The configuration is retained across tile invocations.

The configuration-and-execution split establishes a tile-granular interface between the processor and the recurrence engine. Persistent address context supports repeated merge invocations across the workload configurations of Section IV, while the issue adapter serializes requests from the controlling cores into single engine transactions. The completion response is consumed before subsequent state use, providing a definite synchronization boundary without exposing individual LUT or multiplier steps as ISA operations. Vector dimensions and scheduling remain under software control, allowing the scalar engine to integrate with ordinary RVV execution on Spatz.

IV. Evaluation

A. ISA-Controlled Workload Scaling

All main performance measurements use Verilator 5.034 on a two-core Spatz cluster with 128-KiB, 16-bank TCDM, VLEN 512, and four FPUs per core. Software Recurrence + RVV is compared with `OMCFG`/INIT/`OMERGE` Scalar SMU + the same RVV update. Matched cases use the same input tensors, seed, software compilation, and three nested measurement windows: *Recurrence* includes command issue, hardware execution, and completion; *Inclusive Merge* additionally includes RVV update and merge-side orchestration; *Native Core* includes score and tile-state generation, first-tile handling, and final output copying with resident inputs. One-time instruction-context configuration is measured separately and included in the reported Native Core totals; Recurrence and Inclusive Merge retain their directly measured execution windows. Each measured ISA recurrence window spans from command issue through hardware completion synchronization and the dependent status read. The executed instruction streams and RTL response traces confirm the custom-instruction path.

Across nine matched RTL workloads, recurrence/control and inclusive-merge speedups are 51.5–$52.7\times$ and 5.14–$12.08\times$, respectively. Fig. 3 compares configuration-inclusive Native Core cycles (1.024–$1.238\times$). At $N = 16$, increasing the joint Q/K and output dimension D from 32 to 128 reduces Core speedup from $1.238\times$ to $1.070\times$. At $D = 64, B = 4$, increasing square sequence size N from 8 to 32 raises speedup from $1.087\times$ to $1.153\times$. At $N = 32, D = 64$, increasing B from 4 to 16 reduces speedup from $1.153\times$ to $1.024\times$ as merge invocations fall from seven to one.

Across these sweeps, recurrence speedup stays near $52\times$, but the Native Core gain depends on the fraction of work spent in tile merges. Larger D increases score and vector computation outside the SMU, whereas larger B reduces the number of merges. At fixed D and B, larger N increases the query–tile combinations and thus the opportunity for recurrence acceleration.

The raw recurrence acceleration has a different scope from these core results. At $N = 16, D = 64, B = 4$, software needs 68,610 recurrence cycles, whereas the instruction-controlled path needs 1,308 cycles, including 1,248 engine-busy cycles and 60 cycles of recurring command overhead. The once-only setup adds 636 cycles. The engine-busy count corresponds to approximately 26 cycles per processed row for this configuration, as confirmed by the other accepted scaling points. These engine-busy cycles include per-row state traffic and controller sequencing. The same workload's Native Core decreases from 571,415 software cycles to 504,493 configuration-inclusive ISA cycles, a $1.133\times$ speedup. The remaining score-tile formation, local normalization, and RVV updates dominate that larger window. Consequently, a large recurrence speedup translates into a more moderate, workload-dependent kernel improvement.

TABLE II
Matched MMIO-controlled merge ablation (cycles).

$N/D/B$	Scalar+RVV	Full SMU	Full/Scalar
16/64/4	13,100	29,629	2.26×
32/64/8	21,785	55,232	2.54×

The native benchmark uses $d_k = d_v = D$ and square $N_q = N_k = N$ inputs; the D sweep therefore varies score and output work together. In two matched configurations, ISA control and MMIO control produce identical output words. At $16/64/4$, ISA setup and execution require 504,493 Core cycles versus 508,497 with MMIO; at $32/64/8$, the corresponding totals are 1,845,671 and 1,843,758 cycles. The two control interfaces exhibit comparable end-to-end performance, with differences below 1% in both configurations. These results indicate that recurrence specialization and RVV resource reuse account for the primary acceleration. The custom-instruction interface complements these architectural benefits by encapsulating the multi-row recurrence as a tile-level operation with persistent configuration and processor-visible completion.

B. Selective Versus Full Offload

The two-case microkernel comparison uses MMIO control for both implementations because Full SMU requires vector buffer addresses and dimension parameters beyond the nine-field scalar context of `OMCFG`. Each implementation uses a single 64-bit TCDM requester on the accelerator side, enabling a matched comparison of RVV-based and SMU-based vector updates (Table II). Full offload needs 2.26–$2.54\times$ the selective merge cycles. In the Full engine, vector updates occupy approximately 95.9% of busy cycles: its single 64-bit TCDM requester processes output elements sequentially, issuing reads of two source vectors and a write per element, whereas Spatz uses its parallel RVV memory and FPU resources. The observed extra read-response stall count is zero, indicating that serialized vector traffic and request sequencing, rather than memory-response contention, dominate this case. The observed vector phase takes approximately eight engine cycles per output element; the measured vector read/write transactions are 6,144/3,072 and 12,288/6,144, with only 39 and 42 request-grant stall cycles. There is no evidence of a TCDM read-response queue bottleneck in these cases: the transfer serialization is imposed by the Full engine's one-request-at-a-time execution structure. The Full design also increases Native Core cycles by only 3.11% and 1.85%, respectively, because the merge contributes a limited share of total attention-kernel work.

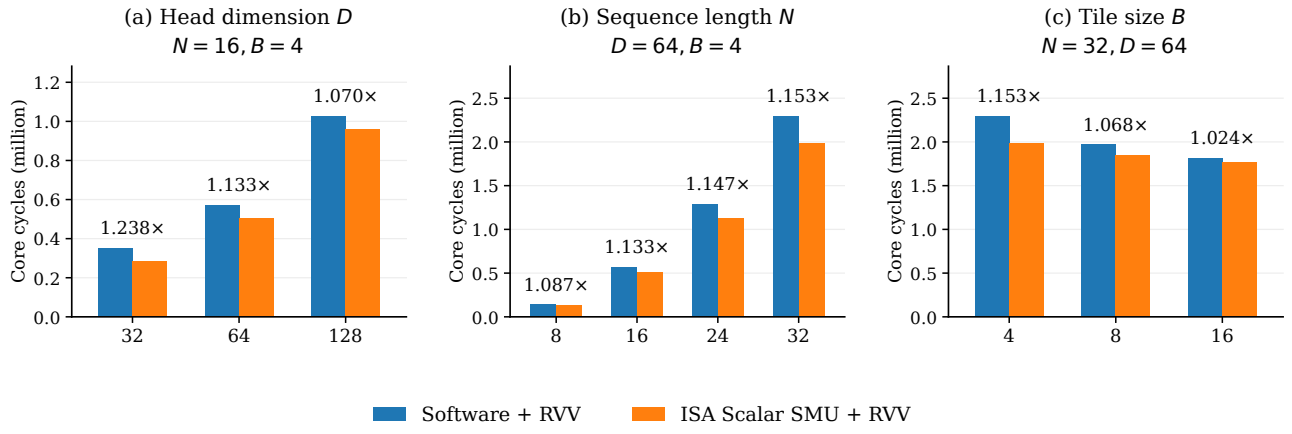


Fig. 3. Native Core cycles under three scans: (a) D at $N = 16, B = 4$; (b) N at $D = 64, B = 4$; (c) B at $N = 32, D = 64$. Bar pairs compare software + RVV with ISA Scalar SMU + RVV (including one-time configuration). Annotations show speedup.

TABLE III
MATCHED TWO-CORE CLUSTER CELL AREA, EXCLUDING 20 SRAM MACROS; NANGATE45 TYPICAL LIBERTY AREA UNITS.

Design	Cell area	Relative
Software + RVV	3,869,821	Baseline
Scalar SMU + RVV	3,931,951	+1.61%

C. Area, Accuracy, and Architectural Positioning

In matched Nangate45 synthesis of the two-core cluster, the scalar-only SMU configuration adds 62,130 mapped standard-cell area units (1.61%) to the software/RVV baseline (Table III). The SMU update engine accounts for 59,847 units of that increment; its command adapter and arbitration logic bring the SMU-related subtree to 60,947 units, or about 98.1% of the overall added logic. The cluster synthesis instantiates the scalar-only mixed-precision engine, while the functional RTL used for the matched Full-SMU ablation also retains the separate full-offload mode. A separate module-level mapping comparison reduces the recurrence arithmetic area from 72,303 units (FP32 reference datapath) to 52,080 units (implemented mixed-precision reciprocal datapath), or 27.97%. The reduction is driven by stage-aware precision rather than reciprocal replacement alone: the mixed-precision division variant maps to 51,612 units.

Across the nine ISA workloads, output MAE against the FP32 reference is 1.91×10^{-4}–5.63×10^{-4}, with stable relative error of 0.282–0.832%; all outputs are finite. Matched ISA/MMIO runs produce identical output words. The one-cycle SCALAR phase in Fig. 2 represents RTL scheduling. Nangate45 synthesis reports mapped standard-cell area excluding SRAM macros; physical timing closure and power are not evaluated.

Table IV locates the contribution among nonlinear and Softmax accelerators. Reuse of nonlinear-function hardware has also been explored for Softmax/GELU functions [15]; the present design instead shares recurrence arithmetic across the two partial states and exploits RVV for the output. Scalar SMU specifically targets the normalized state transition between attention tiles and reuses the cluster's vector arithmetic for the output. Performance is evaluated against matched software and full-offload implementations on the same RTL platform, while mapped-area overhead is measured against the software/RVV cluster baseline.

TABLE IV
FUNCTIONAL COMPARISON WITH RELATED HARDWARE SPECIALIZATION; REPORTED WORKS USE DIFFERENT KERNELS AND PLATFORMS.

Work	Accelerated operation	Execution organization
Softermax [6]	Online Softmax normalization	Dedicated Softmax arithmetic
SoftEx [7]	Softmax / GELU	RISC-V heterogeneous unit
VEXP [8]	Exponential / Softmax	Snitch ISA/FPU extension
This work	Tile-to-tile state recurrence	Cluster SMU + existing RVV

V. CONCLUSION

Scalar SMU implements online attention's dependent normalization recurrence as a shared, row-sequential RISC-V cluster accelerator. Its compact instruction interface exposes tile-level execution while Spatz/RVV retains the dimension-wise update. Across nine ISA-controlled workloads, merge speedup reaches 5.14–12.08× and configuration-inclusive Native Core speedup reaches 1.024–1.238×. Matched full-offload measurements identify serial vector traffic as the dominant added cost; scalar-only cluster synthesis adds 1.61% mapped standard-cell area. The results show how recurrence-specific logic can complement an existing programmable vector datapath.

REFERENCES

[1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," in *Proc. Adv. Neural Inf. Process. Syst. (NIPS)*, vol. 30, 2017, pp. 5998–6008.
[2] M. Milakov and N. Gimelshein, "Online normalizer calculation for Softmax," arXiv preprint arXiv:1805.02867, 2018.
[3] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré, "FlashAttention: Fast and memory-efficient exact attention with IO-awareness," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 35, 2022, pp. 16 344–16 359.
[4] T. Dao, "FlashAttention-2: Faster attention with better parallelism and work partitioning," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2024.

[5] J. Shah, G. Bikshandi, Y. Zhang, V. Thakkar, P. Ramani, and T. Dao, "FlashAttention-3: Fast and accurate attention with asynchrony and low-precision," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 37, 2024, pp. 68 658–68 685.

[6] J. R. Stevens, R. Venkatesan, S. Dai, B. Khailany, and A. Raghunathan, "Softermax: Hardware/software co-design of an efficient Softmax for transformers," in *Proc. ACM/IEEE Des. Autom. Conf. (DAC)*, 2021, pp. 469–474.

[7] A. Belano, Y. Tortorella, A. Garofalo, L. Benini, D. Rossi, and F. Conti, "A flexible template for edge generative AI with high-accuracy accelerated Softmax and GELU," *IEEE J. Emerg. Sel. Topics Circuits Syst.*, vol. 15, no. 2, pp. 200–216, 2025.

[8] R. Wang, G. Islamoglu, A. Belano, V. Potocnik, F. Conti, A. Garofalo, and L. Benini, "VEXP: A low-cost RISC-V ISA extension for accelerated Softmax computation in transformers," in *Proc. IEEE Symp. Comput. Arith. (ARITH)*, 2025, pp. 37–44.

[9] F. Zaruba, F. Schuiki, T. Hoefler, and L. Benini, "Snitch: A tiny pseudo dual-issue processor for area and energy efficient execution of floating-point intensive workloads," *IEEE Trans. Comput.*, vol. 70, no. 11, pp. 1845–1860, 2021.

[10] M. Perotti, S. Riedel, M. Cavalcante, and L. Benini, "Spatz: Clustering compact RISC-V-based vector units to maximize computing efficiency," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 44, no. 7, pp. 2488–2502, 2025.

[11] H. Genc *et al.*, "Gemmini: Enabling systematic deep-learning architecture evaluation via full-stack integration," in *Proc. ACM/IEEE Des. Autom. Conf. (DAC)*, 2021, pp. 769–774.

[12] M. Cavalcante, F. Schuiki, F. Zaruba, M. Schaffner, and L. Benini, "Ara: A 1-GHz+ scalable and energy-efficient RISC-V vector processor with multiprecision floating-point support in 22-nm FD-SOI," *IEEE Trans. VLSI Syst.*, vol. 28, no. 2, pp. 530–543, 2020.

[13] Q.-X. Wu, S.-S. Teng, M.-D. Shieh, C.-T. Huang, and J.-M. Lu, "A low-complexity and reconfigurable design for nonlinear function approximation in transformers," *IEEE Trans. Circuits Syst. II, Exp. Briefs*, vol. 72, no. 12, pp. 1882–1886, 2025.

[14] *The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA*, RISC-V International, 2019, document Version 20191213.

[15] C. Peltekis, K. Alexandridis, and G. Dimitrakopoulos, "Reusing Softmax hardware unit for GELU computation in transformers," in *Proc. IEEE Int. Conf. AI Circuits Syst. (AICAS)*, 2024, pp. 159–163.